

Configuración de la regla en IoT core_AWS_ SQL, Lambda, Dynamo

Para la configuración de la regla iniciamos sesión en la consola y abrimos IoT Core. Una vez dentro en el panel lateral izquierdo buscamos la sección “Administrar”, en administrar damos clic en la opción “enrutamiento de mensajes” y seleccionamos “Reglas” (figura 48).

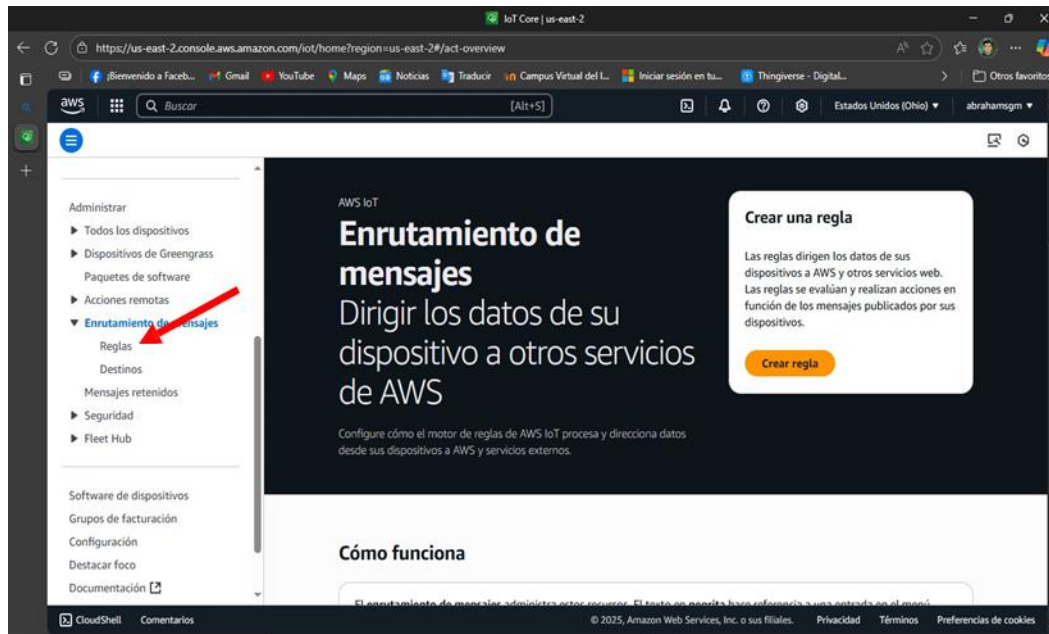


Fig. 1 Enrutamiento de mensajes

Al ingresar en reglas aparecerá la siguiente pantalla, para nuestro caso damos clic en crear regla (figura 49), esto iniciara un gestor de configuración de regla, debemos de seguir los pasos que se muestran en pantalla.

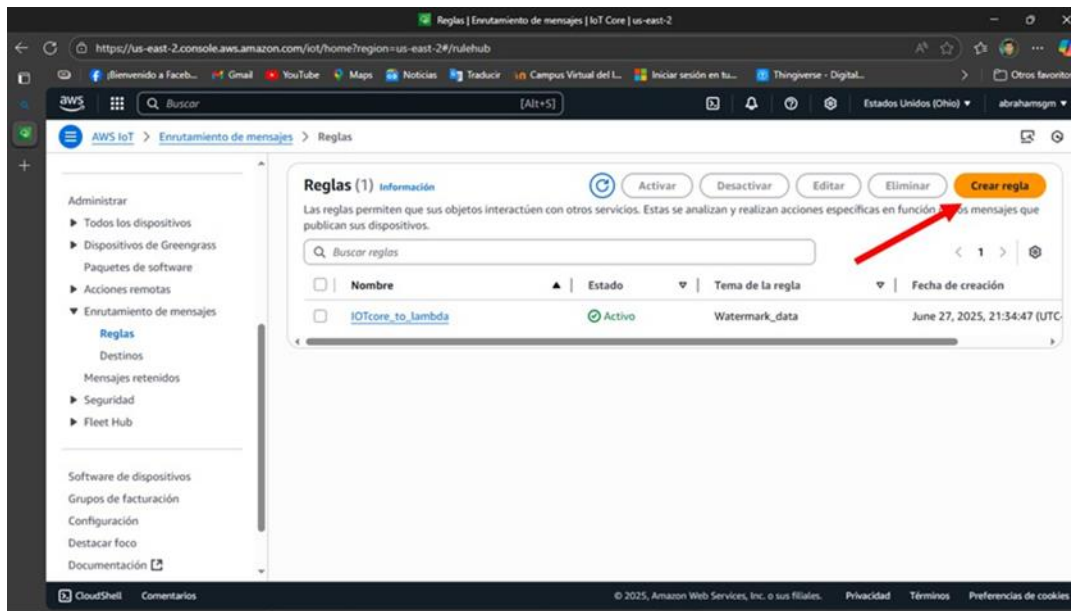


Fig. 2 Crear regla

En el paso 1 pedirá adjuntar el nombre a nuestra regla, esto ayudará a identificarla en los pasos posteriores. Damos de nombre “IoT_to_Lambda”, también podemos agregar una breve descripción que permita entender el comportamiento de la misma (figura 50) .

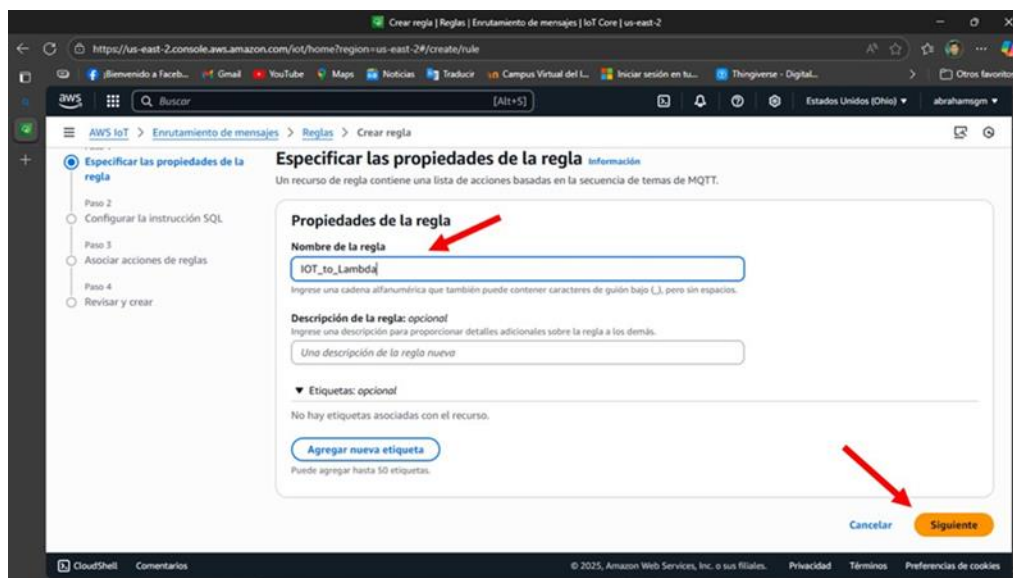


Fig. 3 Propiedades de la regla

En el paso numero 2 configuramos la instrucción SQL, la instrucción SQL permite filtrar los mensajes publicados en los tópicos. Se programa usando la sintaxis de SQL.

Recordemos que los datos del sensor se publican en el tópico “Watermark_data”, también los datos recibidos están en formato JSON siguiendo el siguiente formato:

```
{“Date”:”dd/mm/aa”,  
“Time”:”hh/mm/ss”,  
“sensor_ID”:”WTM1”, “sensor_Read”:”199”  
}
```

Es necesario elegir todas las claves del mensaje JSON publicado en el tópico “Watermark_data”, para eso definimos la instrucción SQL de la siguiente manera(figura 51).

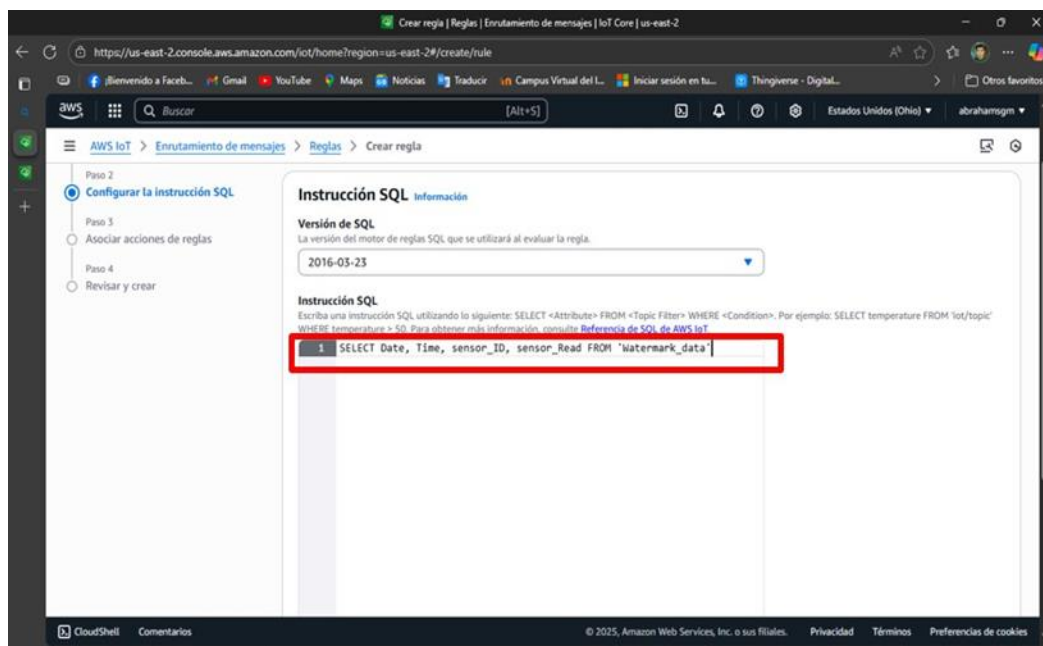


Fig. 4 Instrucción SQL de la regla

Una vez agregada la instrucción SQL que rige la regla, damos clic en siguiente. Nos mostrara un panel donde se debe de configurar la acción que realizara nuestra regla (figura 52), recordemos que los mensajes publicados en el broker deben de ser enviados a AWS Lambda. Buscamos el servicio y lo agregamos.

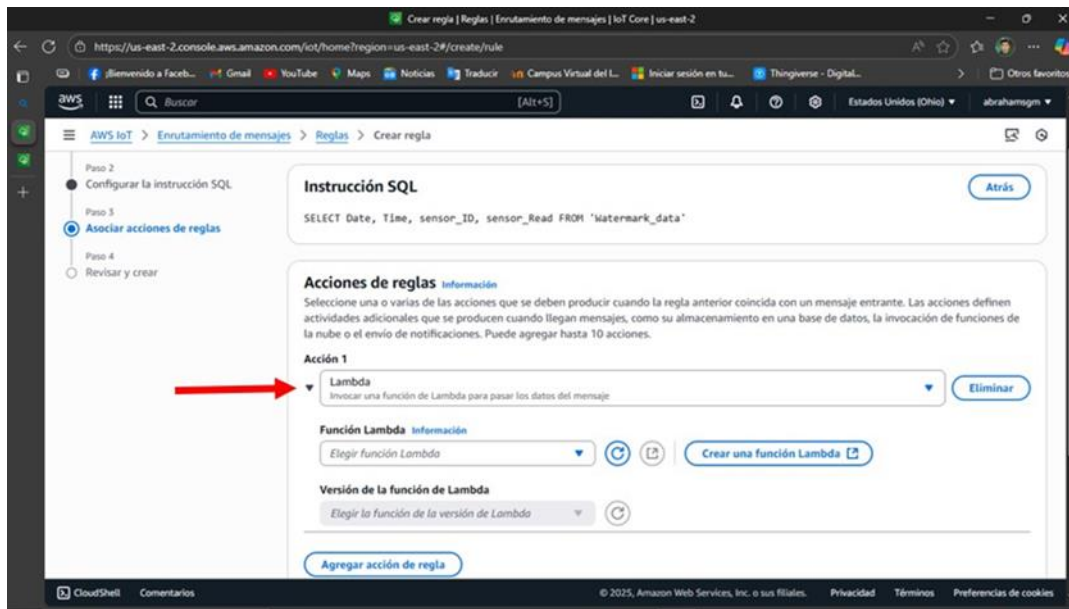


Fig. 5 Acción de la regla

Cuando se cumpla la instrucción demuestra regla se invocará una función lambda para pasar los datos del mensaje. Debemos de agregar la función Lambda que se invocara en nuestro caso creamos una función nueva al dar clic en “Crear función Lambda” (figura 53).

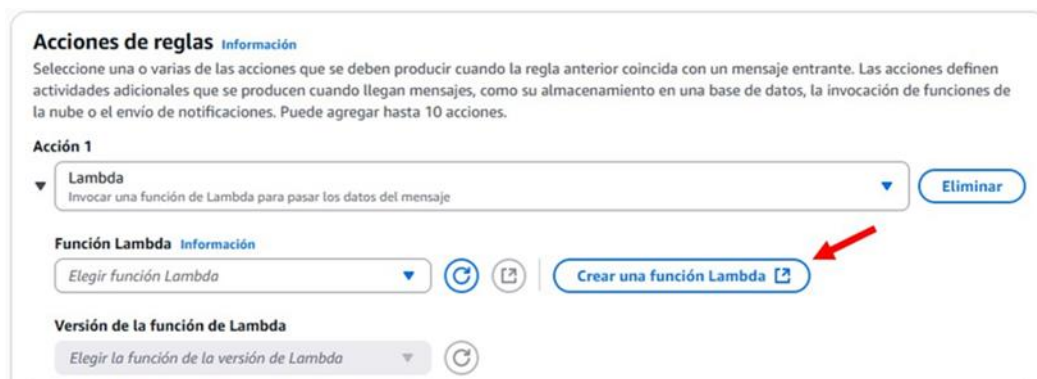


Fig. 6 Crear función Lambda

Al crear una función lambda se abrirá la siguiente ventana de AWS Lambda (figura 54).

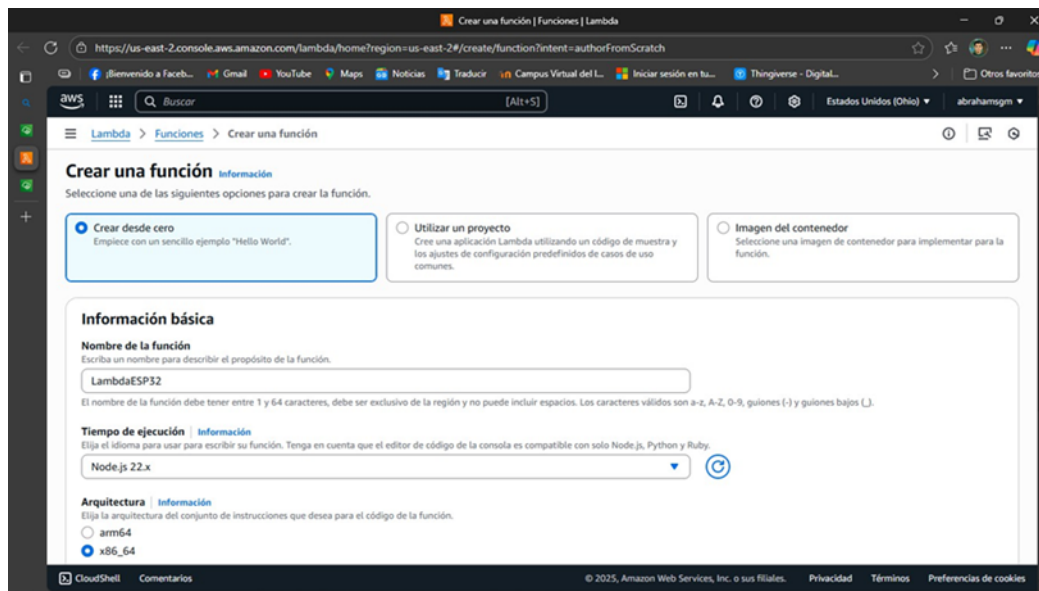


Fig. 7 Creación de una función Lambda desde cero.

Para crear una nueva función Lambda seleccionamos la opción “Crear desde cero” y damos nombre a la función, en nuestro caso agregaremos el nombre de “LambdaESP32”. Damos clic en crear (figura 55).

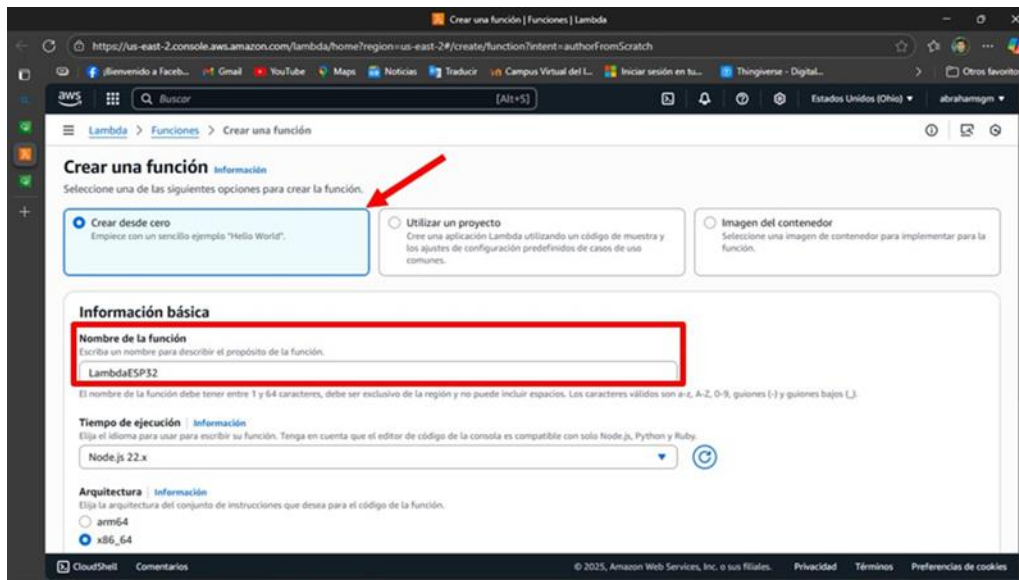


Fig. 8 Configurar nombre de la función Lambda.

Una vez creada nuestra función Lambda regresamos a nuestro gestor de configuración de reglas y en la sección de acciones de reglas buscamos nuestra función Lambda creada, seleccionamos y damos clic en siguiente (figura 56).

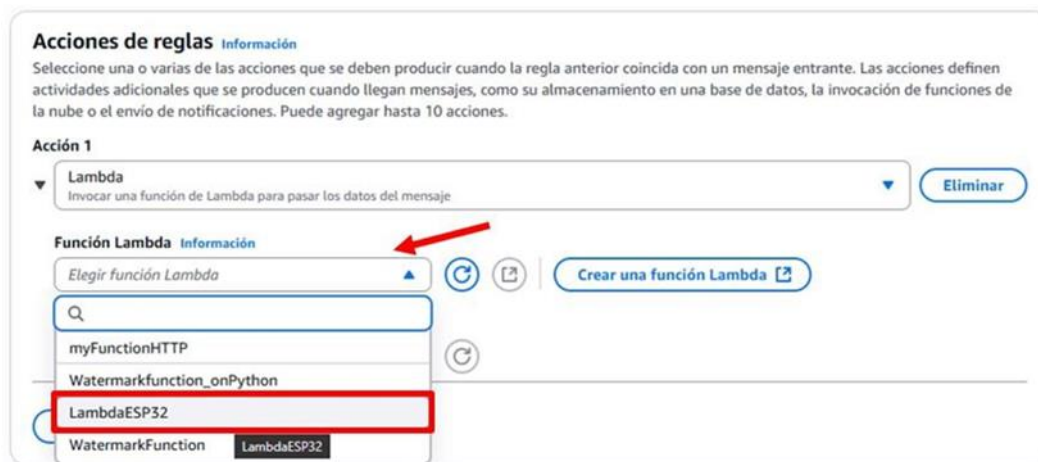


Fig. 9 Configurar acción de la regla.

En el paso 4 se mostrarán las configuraciones elegidas para la regla, debemos de revisar que estas correspondan a las seleccionadas, si todo está bien entonces damos clic en crear (figura 57).

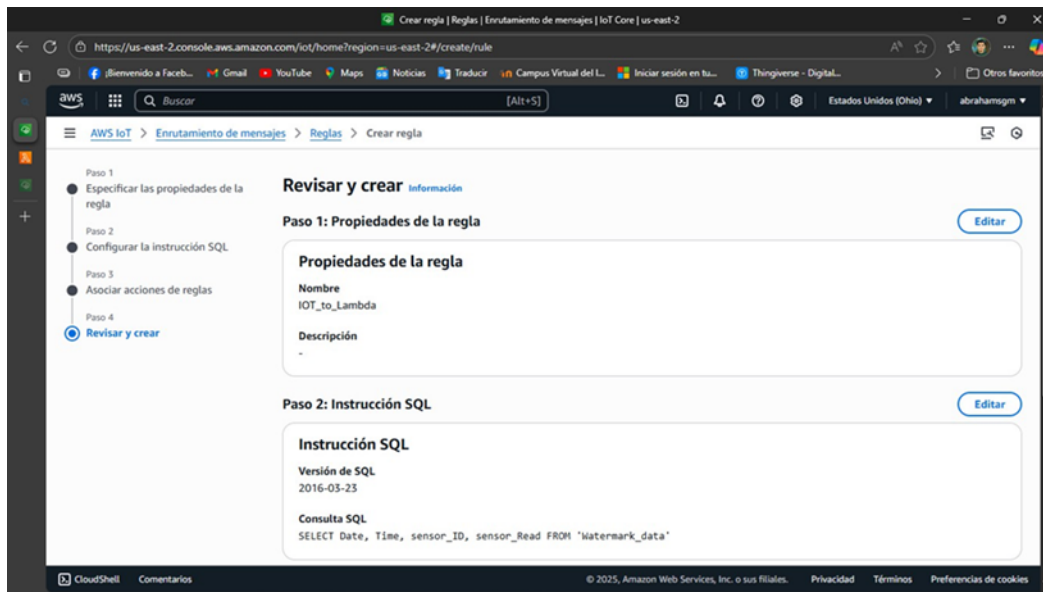


Fig. 10 Revisar y crear regla de enrutamiento

Con estas configuraciones el bróker redireccionara los mensajes publicados en el tópico “Watermark_data” hacia la función “Lambda”, ahora debemos de proceder con las configuraciones de la función Lambda.

Para configurar nuestra función Lambda debemos de ingresar al servicio desde la consola de AWS (figura 58).

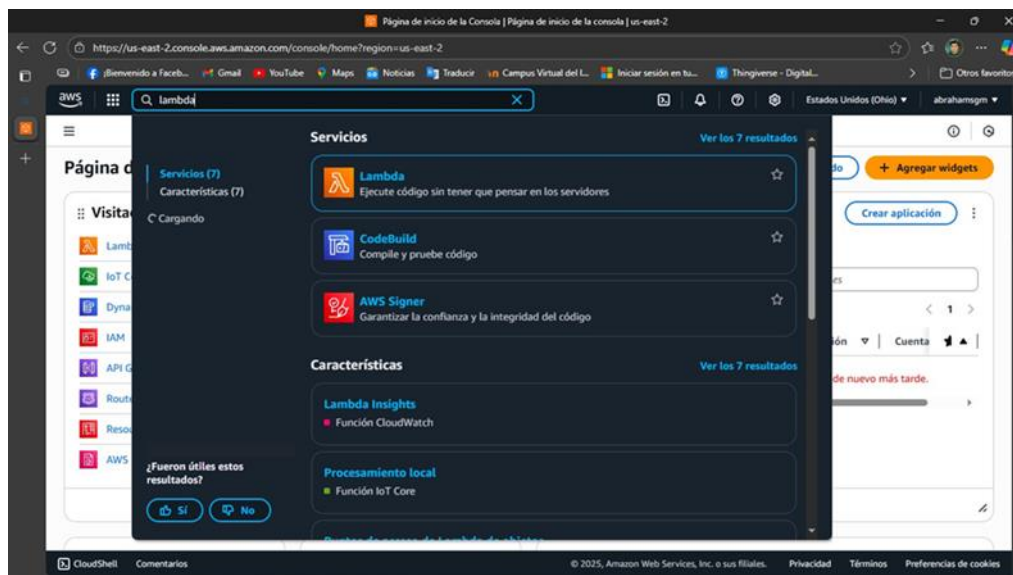


Fig. 11 AWS Lambda.

Al ingresar a AWS Lambda se mostrará un panel como el siguiente, debemos de ubicar la función creada al momento de configurar la regla del broker y dar clic en ella (figura 59).

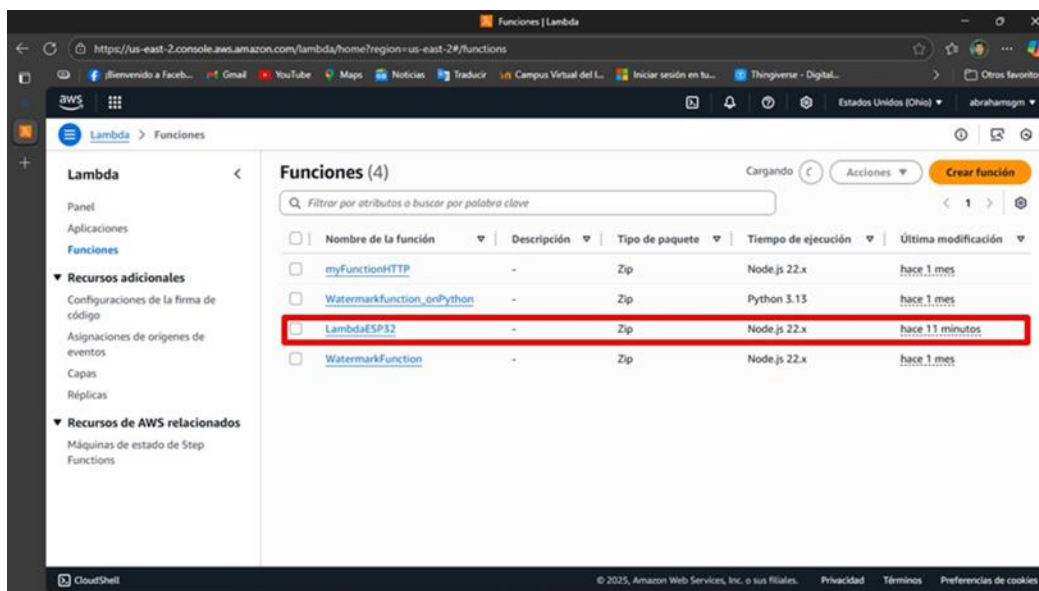


Fig. 12 Ingresar a función creada

Esto nos mostrara el panel de la función Lambda “LambdaESP32” en el que podremos configurar, codificar, monitorear, probar y acceder a aspectos básicos de la función (figura 60).

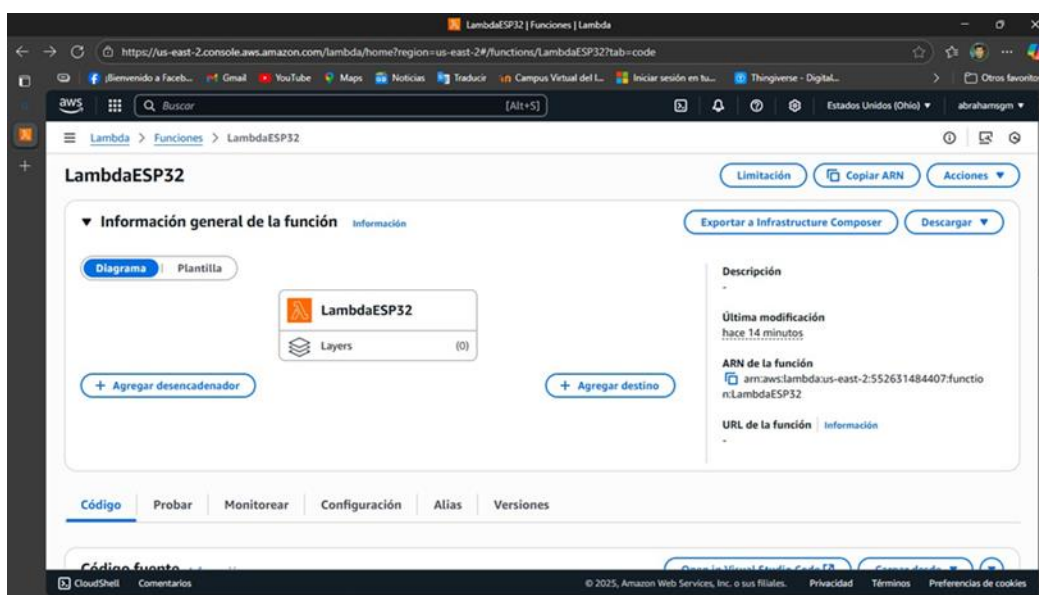


Fig. 13 Panel de la función Lambda creada

El primer paso es establecer el servicio que invocara a nuestra función para esto damos clic en “Agregar desencadenador” (figura 61).

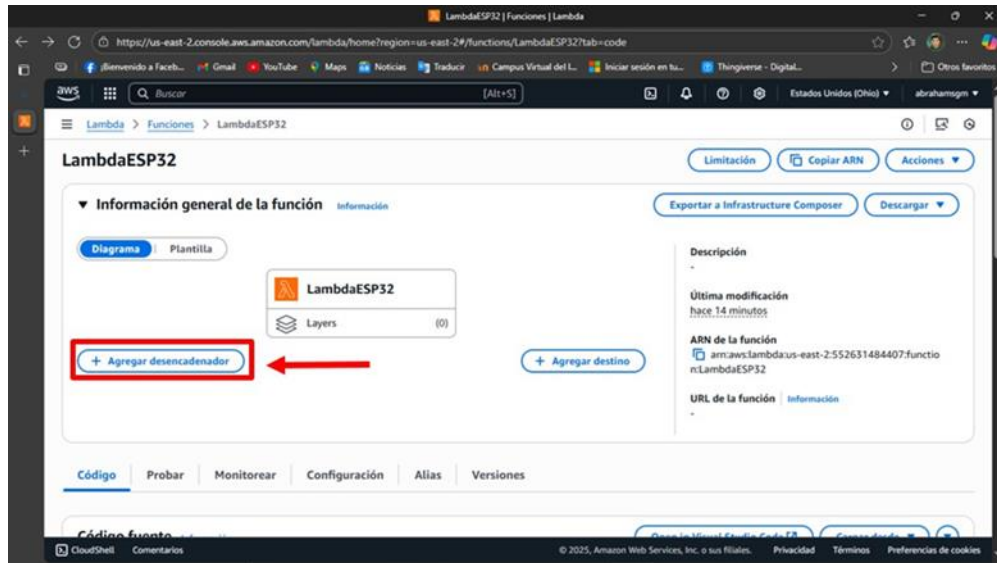


Fig. 14 Agregar desencadenador.

Al dar clic en “Agregar desencadenador” se abrirá la siguiente ventana, en la sección de “Configuración del desencadenador” buscamos el servicio AWS IoT, automáticamente nos mostrará las diferentes opciones en las que AWS IoT puede activar la función Lambda, seleccionamos “Regla de IoT personalizada”, después “Reglas existentes” y en el recuadro de búsqueda seleccionamos la regla creada en los pasos anteriores. Al finalizar damos clic en Agregar (figura 62).

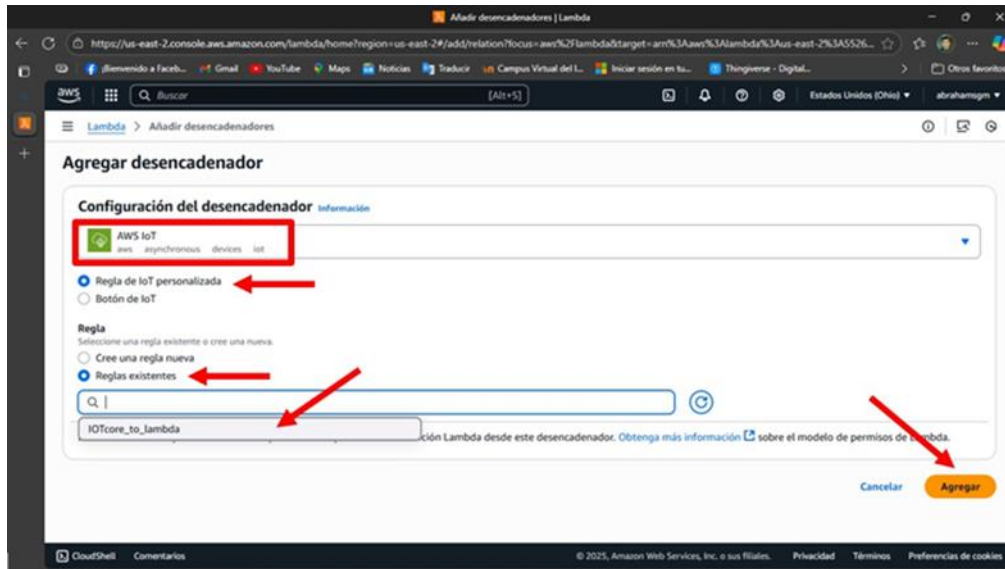


Fig. 15 Configurar AWS IoT como desencadenador.

La configuración de la regla en IoT Core y el desencadenador de la función Lambda garantiza el correcto enrutamiento de nuestros mensajes desde el broker hasta la función.

Cuando la función Lambda se invoca esta ejecutara el código establecido en su configuración. Es posible programar la función en lenguajes como JavaScript, Python, Java, GO o C#. Para establecer el código de nuestra función nos vamos a la sección de “Código fuente”. Para esta aplicación se usará JavaScript (figura 63).

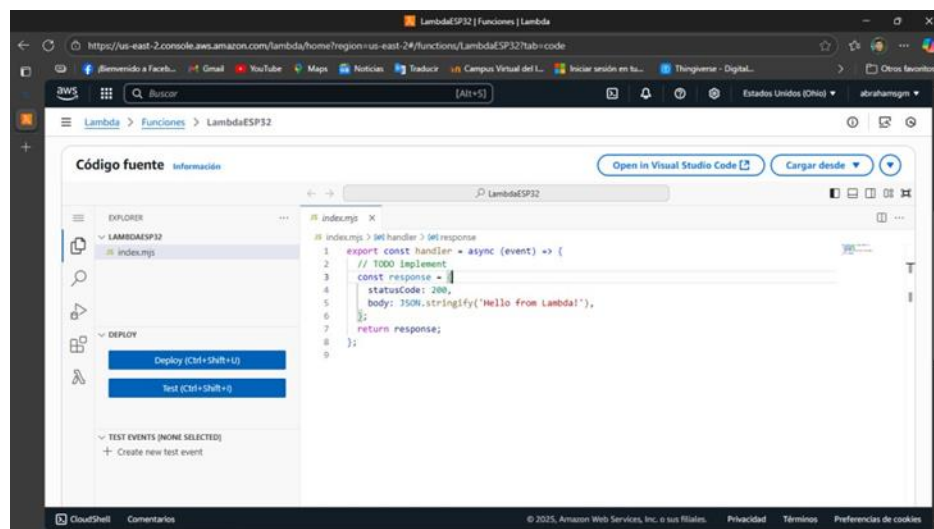


Fig. 16 Código de ejecución de la función Lambda

Se busca que los datos transmitidos a la función Lambda se manden en forma de objeto a DynamoDB de esta forma cada mensaje será registrado en la base de datos como un ítem.

Cada Ítem tendrá los siguientes parámetros: Date, Time, sensor_ID y sensor_Read. A continuación, se muestra el código implementado en la función Lambda (figura 64).



```
JS index.mjs x
JS index.mjs > ...
1 import { DynamoDBClient } from "@aws-sdk/client-dynamodb";
2 import { PutCommand, DynamoDBDocumentClient } from "@aws-sdk/lib-dynamodb";
3
4 const client = new DynamoDBClient({});
5 const docClient = DynamoDBDocumentClient.from(client);
6
7
8 // Este es el handler de tu función Lambda que AWS invocará.
9 export const handler = async (event) => {
10
11     const command = new PutCommand({
12         TableName: "Watermark_data",
13         Item: {
14             Date: event.Date,
15             Time: event.Time,
16             sensor_ID: event.sensor_ID,
17             sensor_Read: event.sensor_Read,
18         },
19     });
20     const response = await docClient.send(command);
21     console.log(response);
22     // 7. Retorna la respuesta.
23     return response;
24 };
```

Fig. 17 Código implementado en función Lambda.

Al principio, se importan las clases necesarias del SDK de AWS para JavaScript. El SDK de Java Script nos permite interactuar con los servicios de AWS. Se usa `DynamoDBClient` para establecer la conexión con DynamoDB, y `PutCommand` junto con `DynamoDBDocumentClient` para realizar las operaciones de inserción de datos en la tabla de DynamoDB. `DynamoDBDocumentClient.from(client)` crea un cliente "documento" que facilita el manejo de ítems de DynamoDB como objetos JavaScript sencillos. Recordemos que los datos recibidos a través del broker cuentan con un formato JSON, es decir tienen notación de un objeto de java script.

La parte principal es el handler, que es la función que AWS invoca cada vez que la función Lambda se activa al llegar un mensaje de la regla de IoT Core. Dentro del handler, se crea un `PutCommand`. Este comando le dice a DynamoDB que se insertara un nuevo ítem en la tabla especificada por el nombre `TableName: "Watermark_data"`. La sección `Item` define los datos que se van a guardar, accediendo directamente los pares clave-valor que se esperan recibir en el event (el JSON que viene de IoT Core). Se espera que el event contenga campos

como Date, Time, sensor_ID, y sensor_Read, los cuales se asignan directamente a las columnas (atributos) del ítem en la tabla de DynamoDB.

Finalmente, `await docClient.send(command)` envía la solicitud a DynamoDB y espera a que la operación se complete. `console.log(response)` permite realizar la depuración de la ejecución del código de nuestra función, mostrando la respuesta de DynamoDB. La función retorna esa misma respuesta, indicando el éxito o fallo de la operación inserción de datos en la tabla “Watermark_data” de DynamoDB.

Cuando se establece el código de nuestra función es necesario darlo de alta, de esta forma nuestra función sabrá que código debe de ejecutar para eso damos clic en el recuadro de “Deploy” (figura 65).

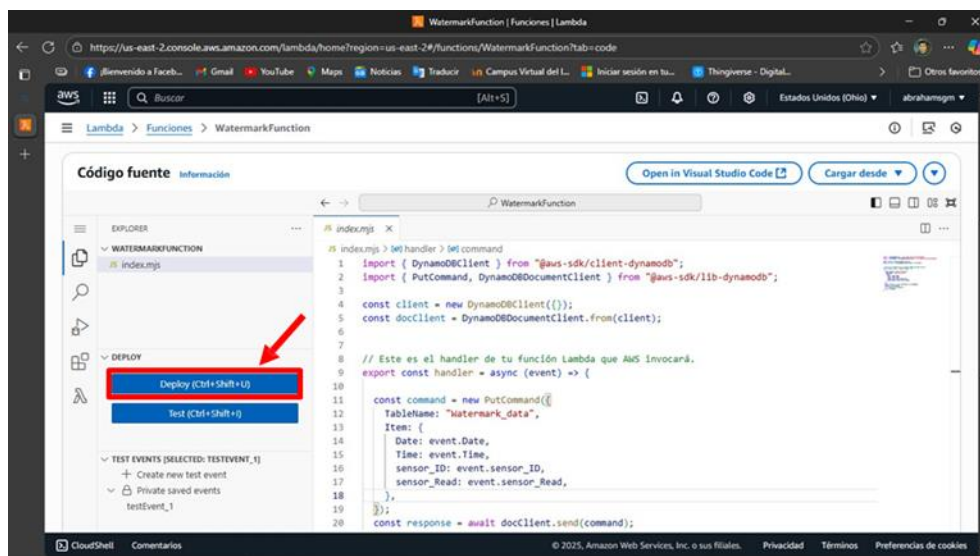


Fig. 18 Desplegar código de la función

Con la función Lambda creada nuestra información será guardada en DynamoDB, ahora debemos de crear la tabla en la que se almacenará, recordemos que el nombre de la tabla debe de corresponder con el definido en el código de la función, para ellos creamos una tabla de nombre “Watermark_data” en DynamoDB.

Accedemos a DynamoDB desde la consola de AWS. Al ingresar se mostrará un panel con lo siguiente para crear tabla (figura 66).

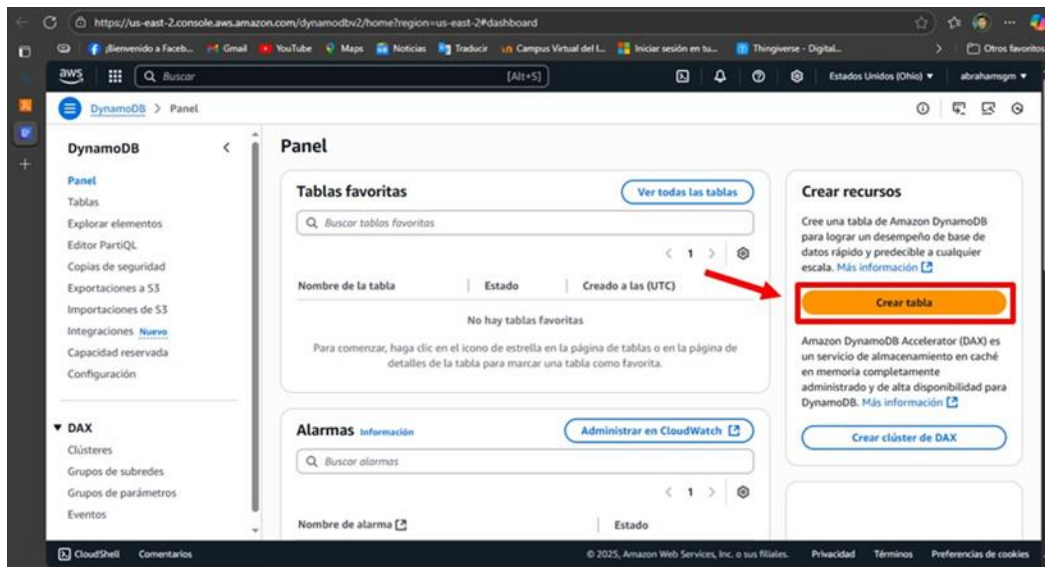


Fig. 19 Crear tabla en DynamoDB

Al dar clic en crear tabla aparecerá la siguiente ventana (figura 67).

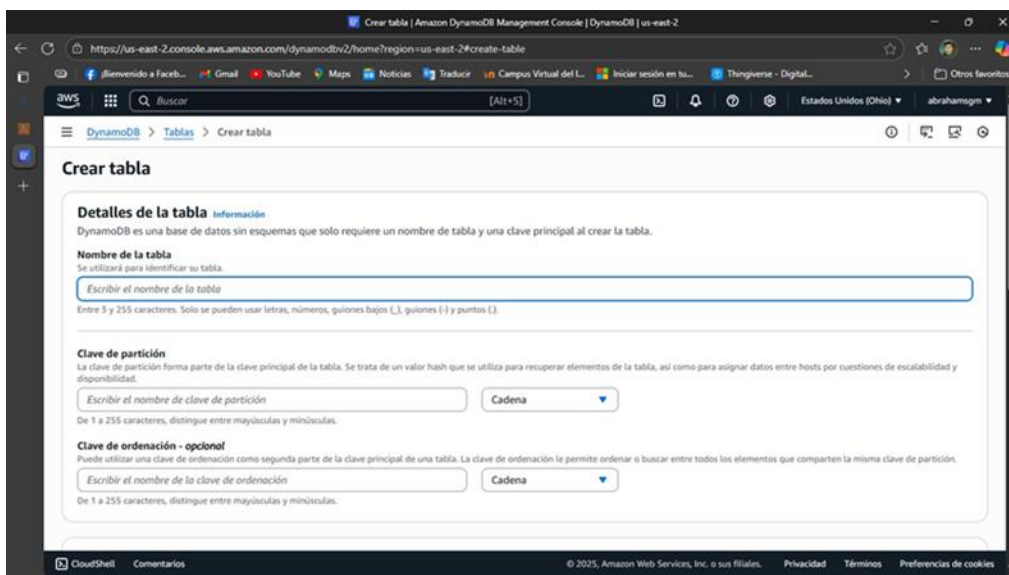
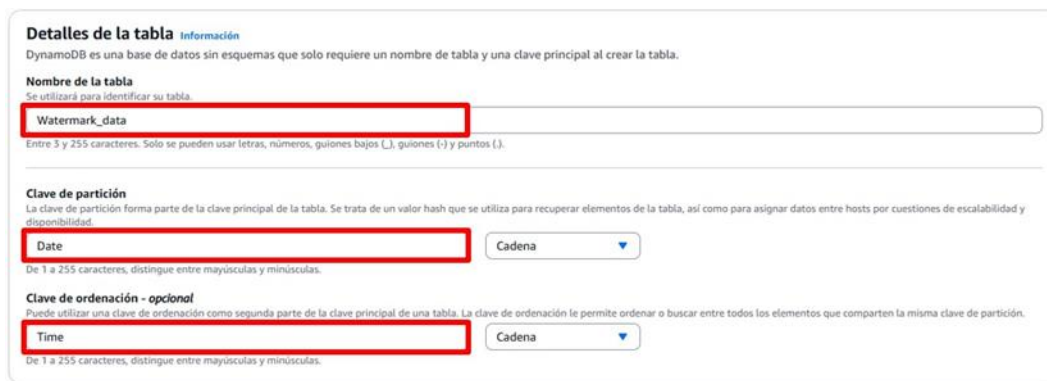


Fig. 20 Panel de creación de tabla

En la sección de detalles de tabla, debemos agregar el nombre de la tabla, clave de partición y clave de ordenación.

El nombre de la tabla será el identificador de la misma; al hacer referencia a él, se podrá acceder a la tabla desde otros servicios de AWS. Establecemos el nombre de la tabla como “Watermark_data”. La clave de partición es una clave que permite identificar y acceder a cada ítem de nuestra tabla. En caso de que uno o más elementos compartan la misma clave de partición, se debe hacer uso de una clave de ordenación. Para la clave de ordenación y partición, se hará uso de los parámetros “Date” y “Time” del ítem agregado a la tabla (figura 68)..



Detalles de la tabla información

DynamoDB es una base de datos sin esquemas que solo requiere un nombre de tabla y una clave principal al crear la tabla.

Nombre de la tabla
Se utilizará para identificar su tabla.

Watermark_data

Entre 3 y 255 caracteres. Solo se pueden usar letras, números, guiones bajos (_), guiones (-) y puntos (.).

Clave de partición
La clave de partición forma parte de la clave principal de la tabla. Se trata de un valor hash que se utiliza para recuperar elementos de la tabla, así como para asignar datos entre hosts por cuestiones de escalabilidad y disponibilidad.

Date

De 1 a 255 caracteres, distingue entre mayúsculas y minúsculas.

Cadena

Clave de ordenación - opcional
Puede utilizar una clave de ordenación como segunda parte de la clave principal de una tabla. La clave de ordenación le permite ordenar o buscar entre todos los elementos que comparten la misma clave de partición.

Time

De 1 a 255 caracteres, distingue entre mayúsculas y minúsculas.

Cadena

Fig. 21 Configurar detalles de la tabla

Hasta este punto, habremos configurado los aspectos básicos para la conexión entre la función Lambda y DynamoDB para el almacenamiento de datos. Sin embargo, la interacción entre servicios de Amazon Web Services se hace a través de roles y políticas. Las políticas establecen los permisos que se pueden llevar a cabo sobre un servicio de AWS; por lo tanto, se debe de configurar una política que permita a la función Lambda acceder a la tabla “Watermark_data” e insertar los ítems. De la misma forma, debemos configurar una política en la tabla de DynamoDB que le dé permisos a la función de Lambda “LambdaESP32” de realizar las operaciones de inserción, logrando que ambas políticas estén referenciadas entre sí.

Las funciones Lambda no están activas siempre, sino únicamente cuando se invocan y ejecutan. Cuando una función se ejecuta, entra en juego su rol de ejecución. Los Roles son una identidad que se puede crear y que tiene permisos específicos con credenciales que son

válidas por periodos cortos. Los Roles son administrados por el servicio de Amazon Web Services IAM. Un rol permitirá a la función Lambda interactuar con DynamoDB.

Para establecer los permisos de la política del rol de ejecución de la función Lambda “LambdaESP32” que permitan insertar ítems en la tabla “Watermark_data” de DynamoDB debemos de acceder al panel de nuestra función “LambdaESP32” en AWS Lambda (figura 69,70).

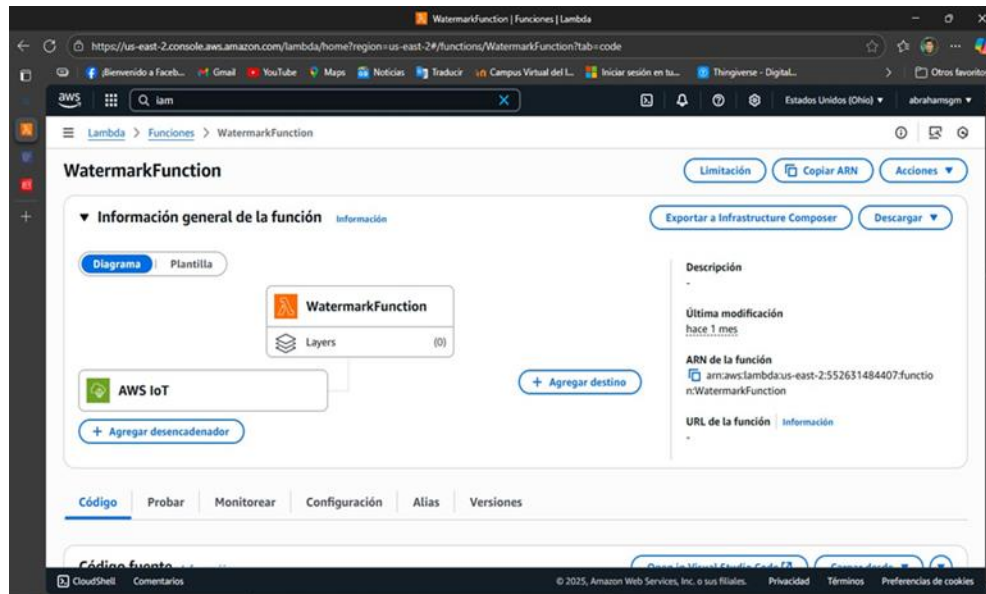


Fig. 22 Función Lambda creada

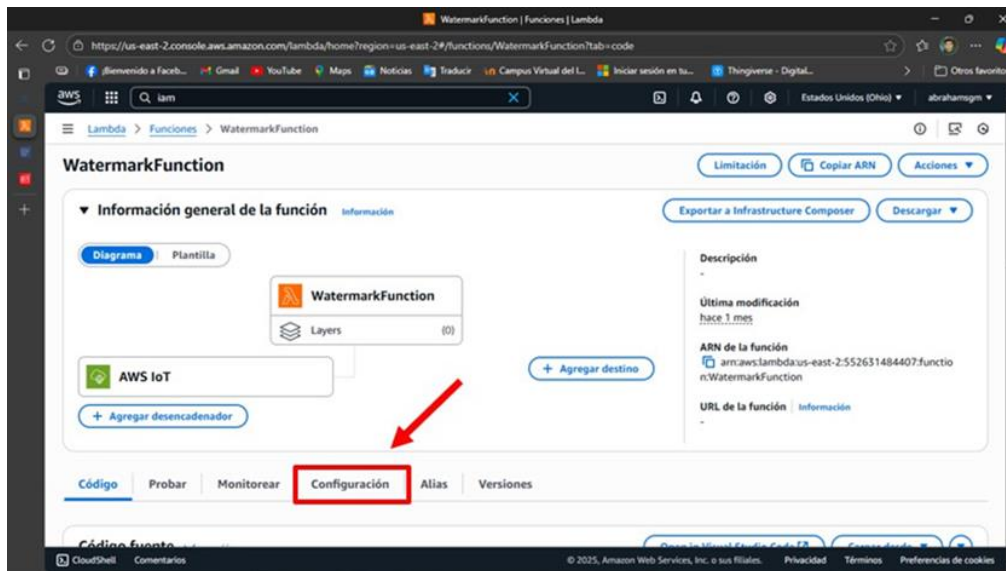


Fig. 23 Sección de configuración de la función Lambda.

En configuración, buscamos la opción de “Permisos” y damos clic en ella. En una ventana aparecerá el rol de ejecución y damos clic en él (figura 71).

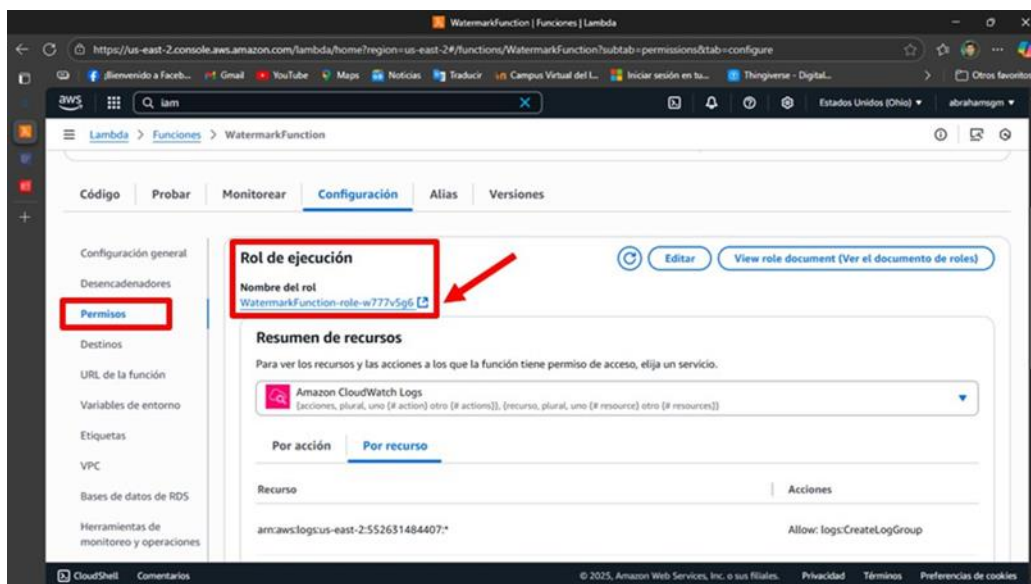


Fig. 24 Rol de ejecución de la función Lambda.

Al haber dado clic en el rol de ejecución, se abrirá una nueva pestaña en la que se mostrará el rol de ejecución de la función “LambdaESP32” dentro AWS IAM (figura 72).

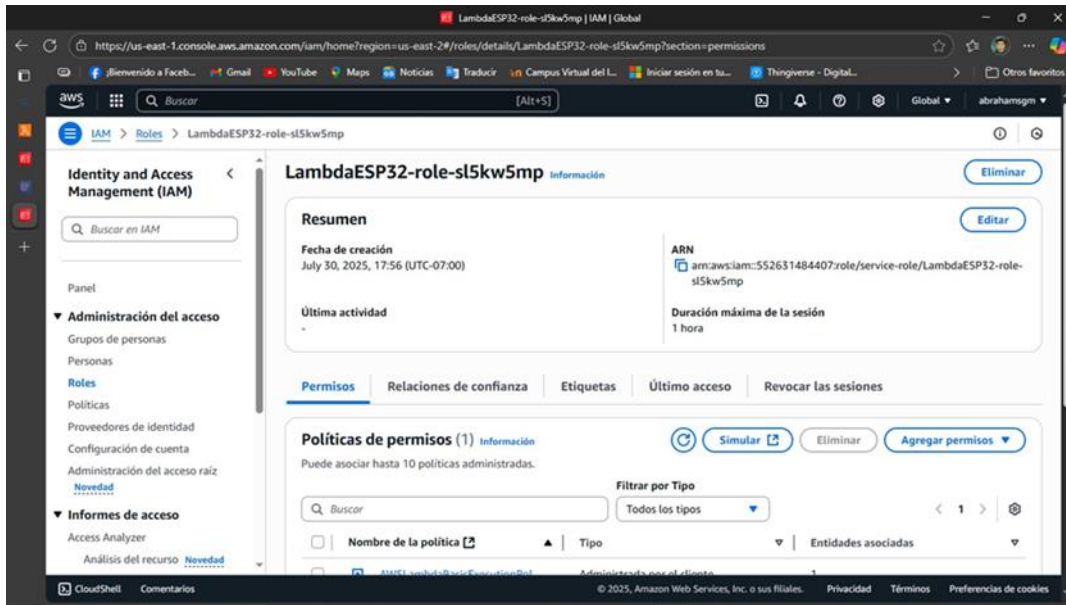


Fig. 25 Panel del rol de ejecución de la función lamdaESP32.

Dentro de la ventana de rol de ejecución buscamos la pestaña de “Políticas de permiso” (figura 73,74).

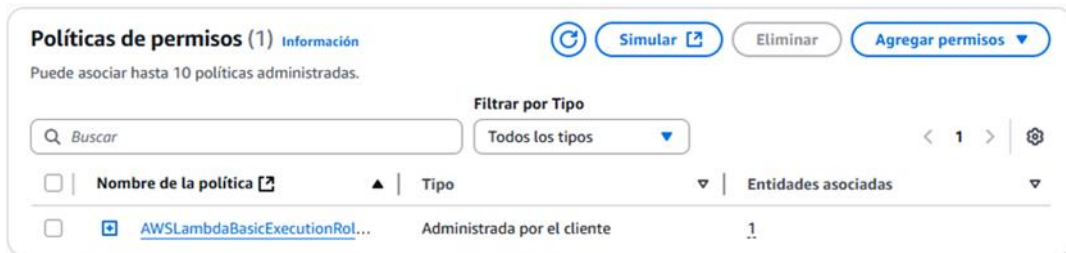


Fig. 26 Políticas de permisos lamdaesp32

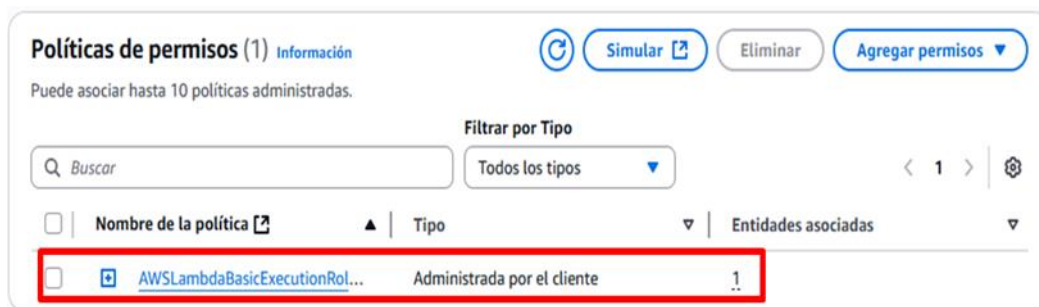


Fig. 27 Política administrada por el cliente AWSLAMDAESP32

Se abrirá una nueva ventana con la política asociada a nuestro rol de ejecución (figura 75)

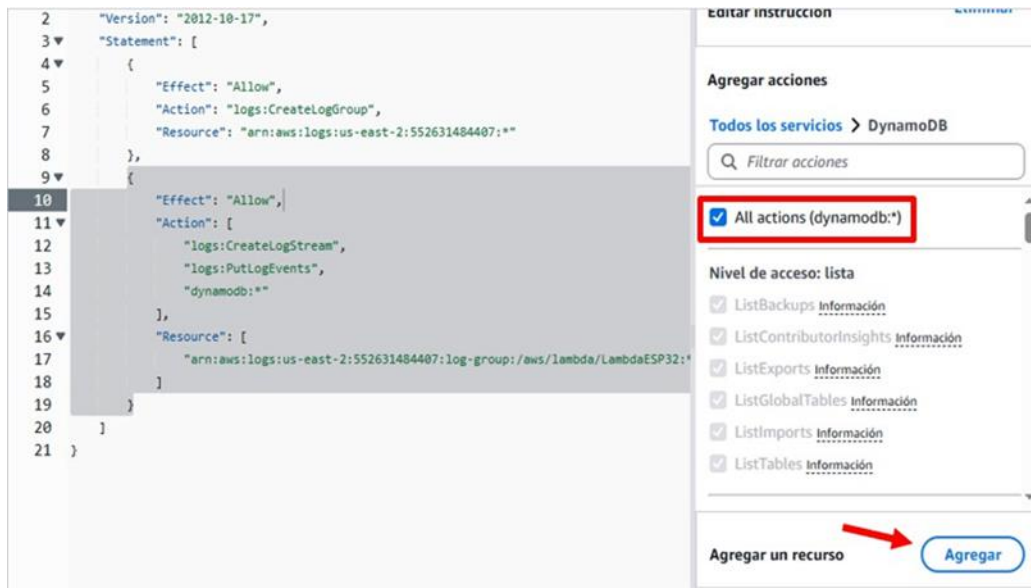


Fig. 29 Agregar acciones de DynamoDB

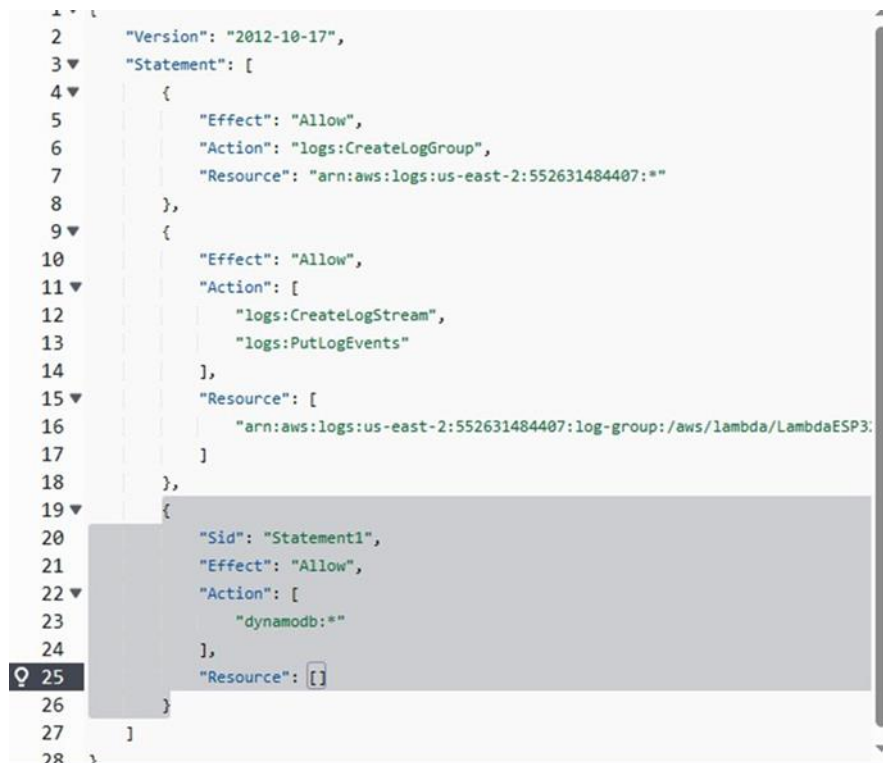


Fig. 30 Recurso Agregado a la política.

En “Resource” (figura 77) debemos de establecer el recurso de nuestra tabla de DynamoDB. abrimos DynamoDB y seleccionamos la tabla “Watermark_data” (figura 78).

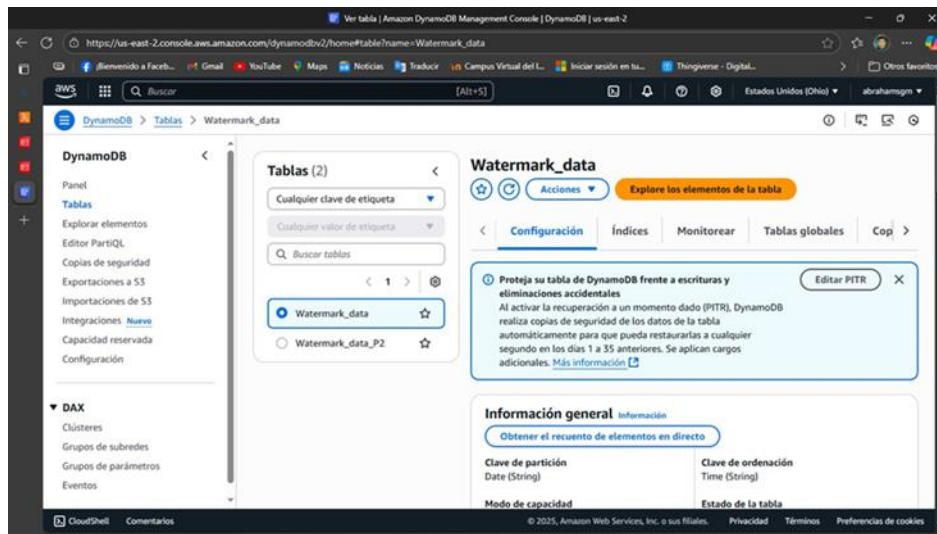


Fig. 31 Panel de inicio de la tabla creada

En la sección de “Información general” aparecerá el recurso de la tabla Watermark_data”, seleccionamos y copiamos (figura 79).

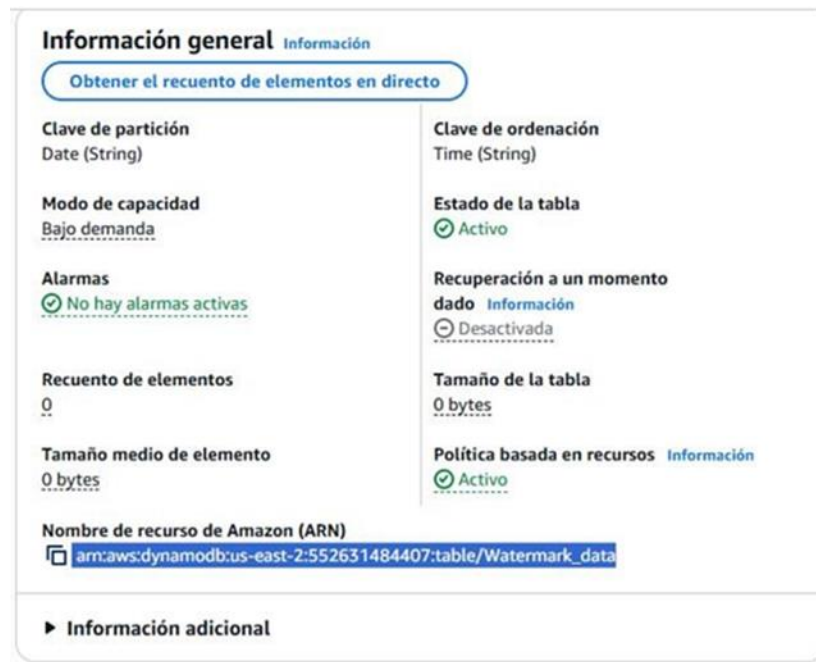


Fig. 32 Nombre del recurso de la tabla creada (watermark data)

Con la configuración de la política del rol de ejecución, la función lambda obtendrá el permiso para acceder a la tabla de datos “Watermark_data” e insertar el ítem con los datos de nuestro sensor. Ahora, debemos configurar la política de la tabla de DynamoDB para que acepte las operaciones del rol de ejecución de la función “LambdaESP32”.

Para configurar la política de la tabla de DynamoDB, accedemos desde el panel a la tabla “Watermark_data”. Dentro de panel de la tabla buscamos y damos clic a la pestaña de “Permisos” (figura 80).

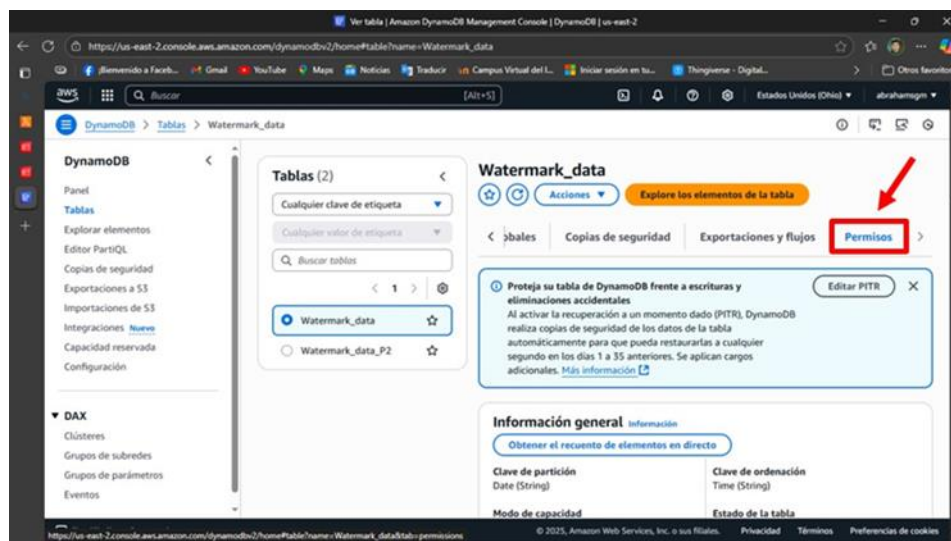


Fig. 33 Configurar permisos de la tabla de DynamoDB

Dentro de permisos buscamos la sección “Política basada en recursos para la tabla”, seleccionamos “Administrar política” y damos clic en “Editar política”.

Ahora se agrega el recurso de nuestro rol de ejecución de la función Lambda. En la pestaña de configuración del panel de la función “LambdaESP32” damos clic en “Permisos” y luego en “Nombre del rol” (figura 81).

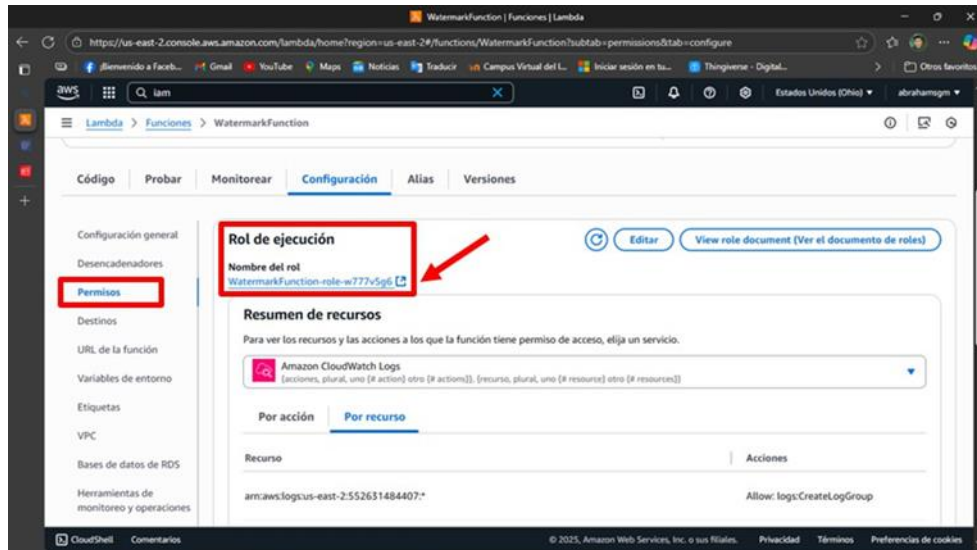


Fig. 34 Rol de ejecución de la función Lambda

Esto abrirá el panel de configuración de nuestro rol de ejecución en AWS IAM. En la sección “Resumen” copiamos el nombre del recurso del rol de ejecución (ARN) y regresamos a la ventana “Agregar entidad principal” de la política de la tabla “Waermark_data” (figura 82).

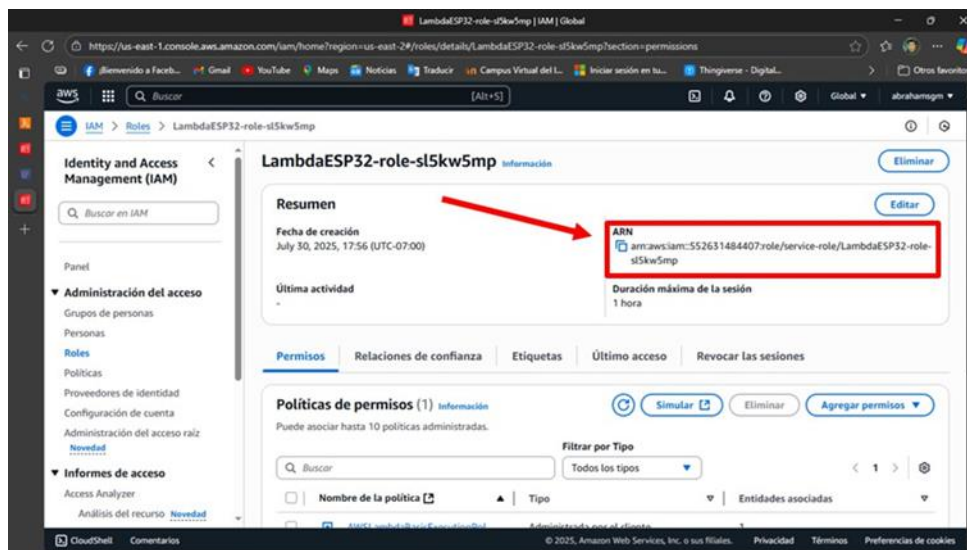


Fig. 35 Nombre del recurso del rol de ejecución (LambdaESP32)

